



Hi! My Name is Hammad Malik 🙌😊

I am a 3rd Year Computer Science student
from **Habib University!** 💻📚

I love buildng web applications! 🌐

Eager to explore how **Bazaar**, Pakistan's leading e-commerce and financial services platform, tackles challenges and delivers value through innovative software solutions.

My hobbies include:

⚽ Playing & watching **football** (40% of the time)

💻 **Coding** & 🐛 ranting about bugs (the rest of it)

Hope u liked my **Pixel Art** (thats literally me) 🎨



>_

```
while(sleeping){  
money++  
}
```

bool internship = True (pls)

I love playing **FIFA** and breaking controllers 🎮

I am a huge **Chai lover** so don't be surprised if you see having my 20th cup of the day 🍵

Head over to this [link](#) to find more about me!

Bazaar Inventory System v1 - Overview

? **Key Question:** How do we track inventory for a single kiryana store effectively?

- Basic Inventory Tracking – Supports stock-in, sales, and removals
- SQLite Database – Lightweight, local storage
- Flask Backend – Simple API + reliable business logic
- Bootstrap Web UI – Clean, responsive interface

Implementation Benefits: Easy deployment, minimal infrastructure, perfect starting point!

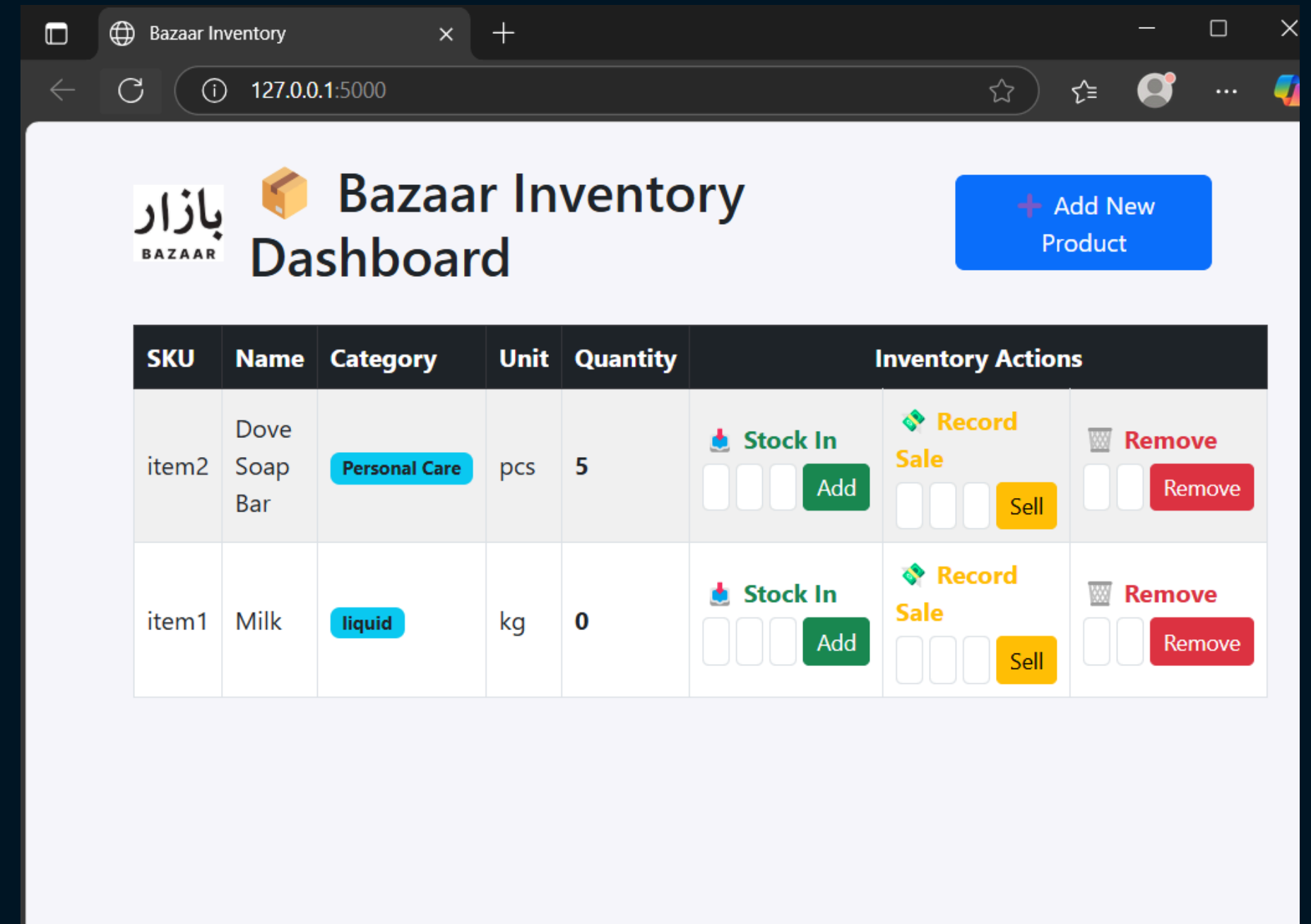


→ **Ideal for small shops with limited tech setup!**

Bazaar Inventory System v1 - Technical Implementation

Features:

- Add & View Products
- Record Stock-In and Sales
- Remove expired/damaged items
- Simple UI, no login required
- Flask : Ensures DB Initialization, Validation & Transaction Safety



Bazaar Inventory System v1 - Architecture & Limitations

? Key Question: When and why should we advance beyond v1?

Because there are **limitations!**

1. Single store support only
2. Limited concurrent user support
3. Basic security model (no authentication 😞)
4. Local-only data access
(Ideally database should be hosted online)



Bazaar Inventory System v2 - Multi-Store Evolution



Key Question: How do we scale from one store to hundreds?

We do this by:

- Central Product Catalog – Shared across all stores
- Store-Specific Inventory – Each store tracks its own stock
- Event-Based Tracking – Every stock change is recorded for full history

Technology Upgrades:

- PostgreSQL – Handles more users + ensures data safety
- JWT Authentication – Secure, role-based access control
- Docker containerization for deployment consistency



Dockerization is crucial!

- Runs the app the same on every machine (no "it works on my computer" issues)
- Bundles everything (code + dependencies) into a container
- Makes scaling and updating much faster

Bazaar Inventory System v2 - API Design & Security



API Question: What's the optimal structure for a multi-store REST API?

RESTful Endpoints:

- Authentication: `/api/v2/auth/*` (login)
- Products: `/api/v2/products/*` (central catalog)
- Inventory: `/api/v2/inventory/*` (store-specific)
- Stores: `/api/v2/stores/*` (management)



Security Features:

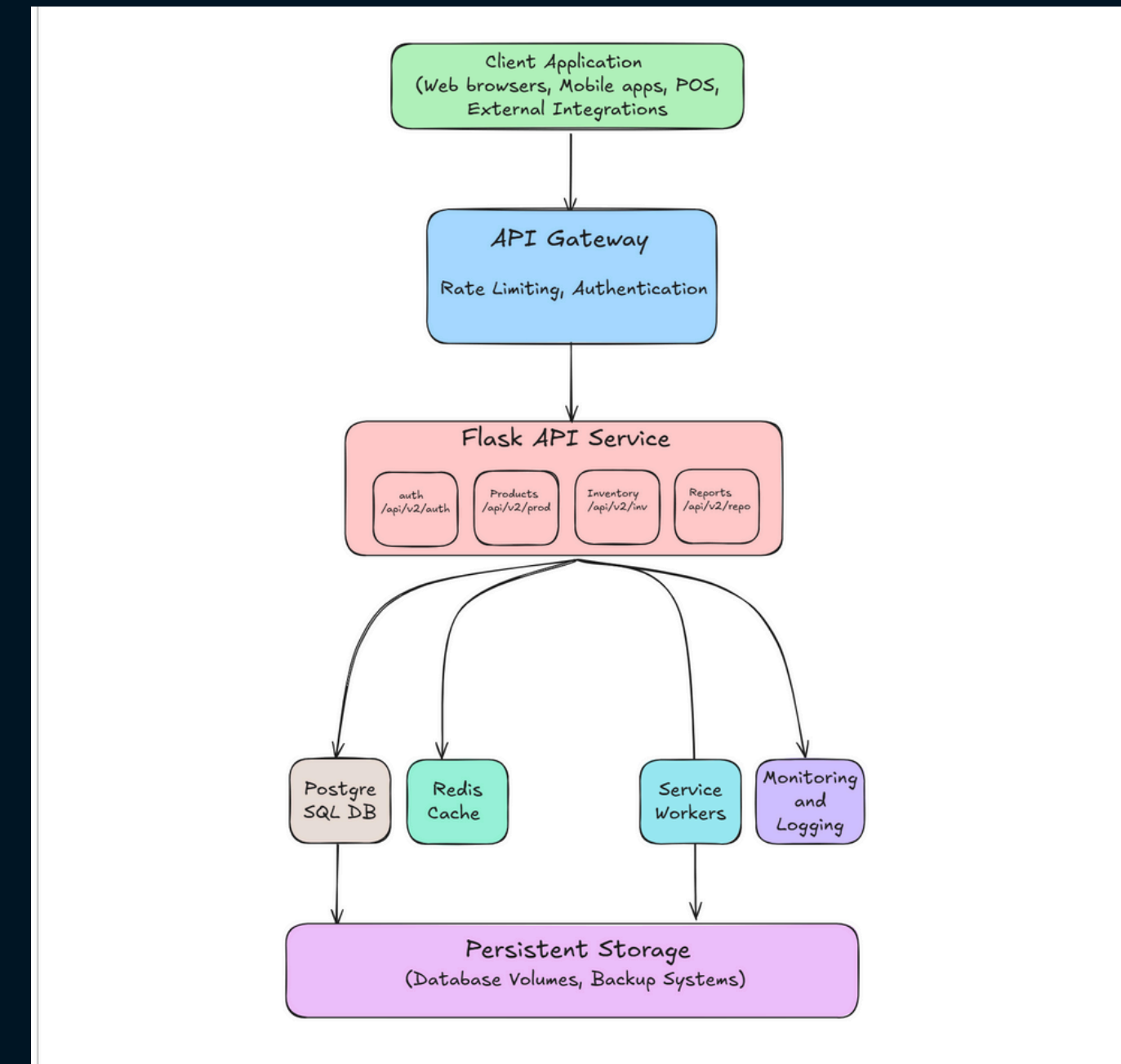
- JWT Authentication – Tokens with role checks
- Store-Based Permissions – Users access only their store's data
- Input Validation & SQL Injection Prevention – Keeps data safe and clean

Bazaar Inventory System v2 - Architecture

Performance Question: How does this architecture handle increased load?

Key Improvements:

- Connection Pooling – Reuses DB connections to reduce overhead
- Docker Compose for streamlined deployment
- Redis Caching – Speeds up access to frequently used data
- Rate Limiting : Good Luck trying to spam requests on an endpoint :P



Bazaar Inventory System v3 – Scaling for Enterprise



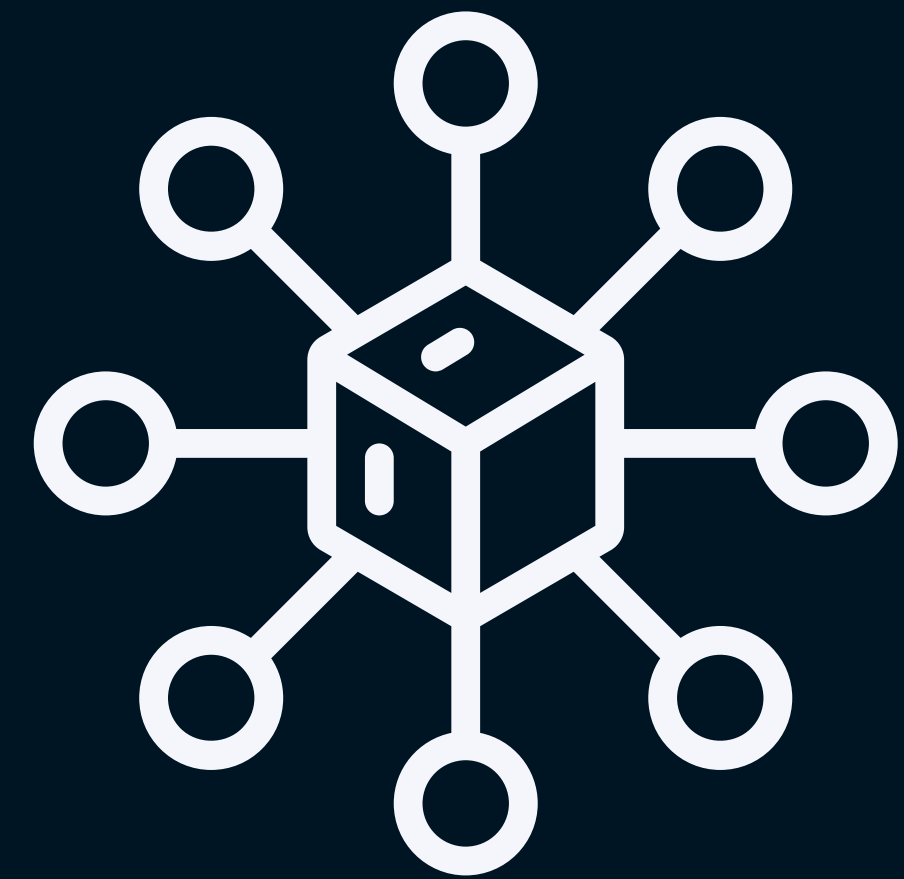
Scaling Question: How do we support thousands of stores with high availability?

Microservices Architecture:

- Independent services communicating via message queues
- Horizontally scalable APIs to handle more traffic
- Background workers for heavy tasks (e.g., reports, sync)

Database Optimization:

- Primary/Replica setup – Writes go to primary, reads to replicas



→ Enables performance, reliability, and scale across thousands of stores

Bazaar Inventory System v3 - Performance Engineering



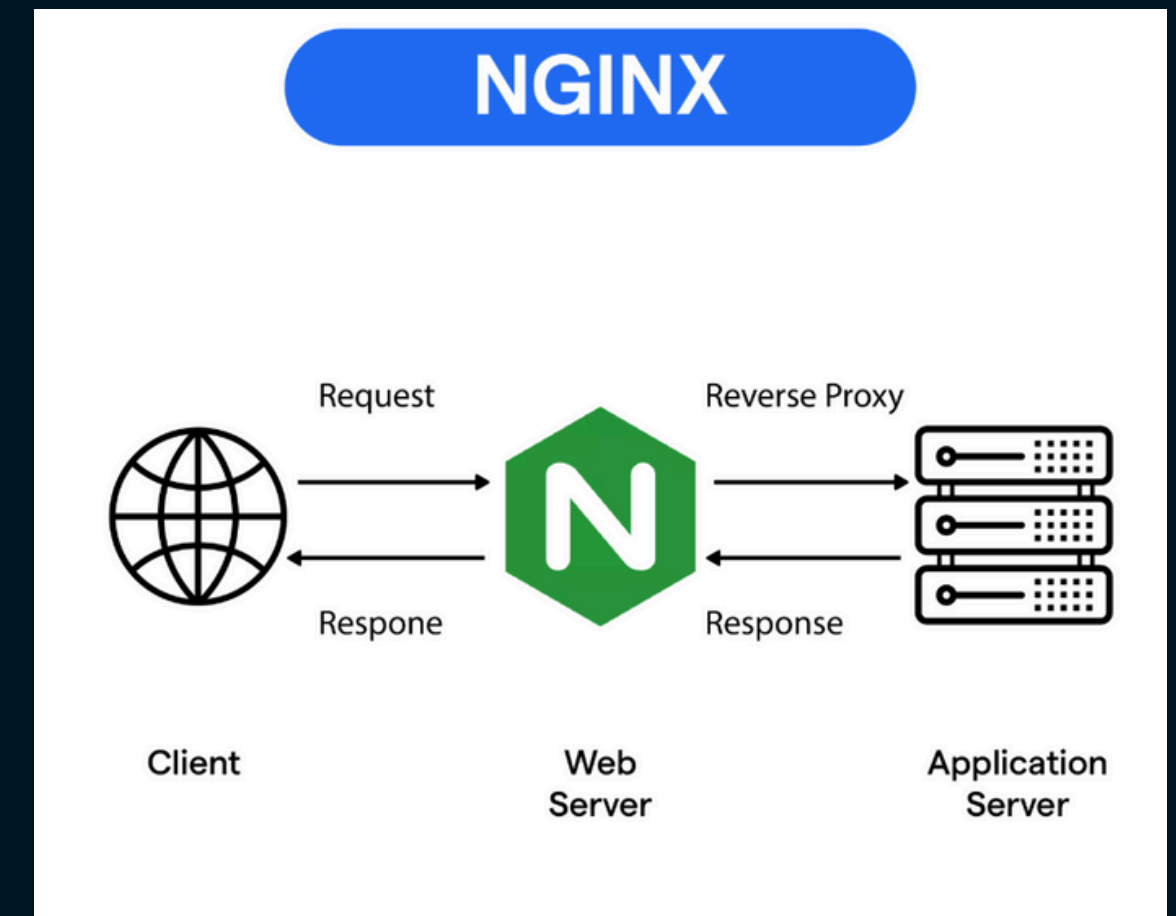
? **Key Question: How do we ensure consistent performance at scale?**

Nginx Load Balancing

- Distributes incoming traffic across multiple API servers
- Built-in health checks ensure requests go only to healthy instances
- Helps prevent overload and ensures high availability

RabbitMQ (Async Processing)

- Offloads heavy tasks (e.g., email, reporting) to background workers



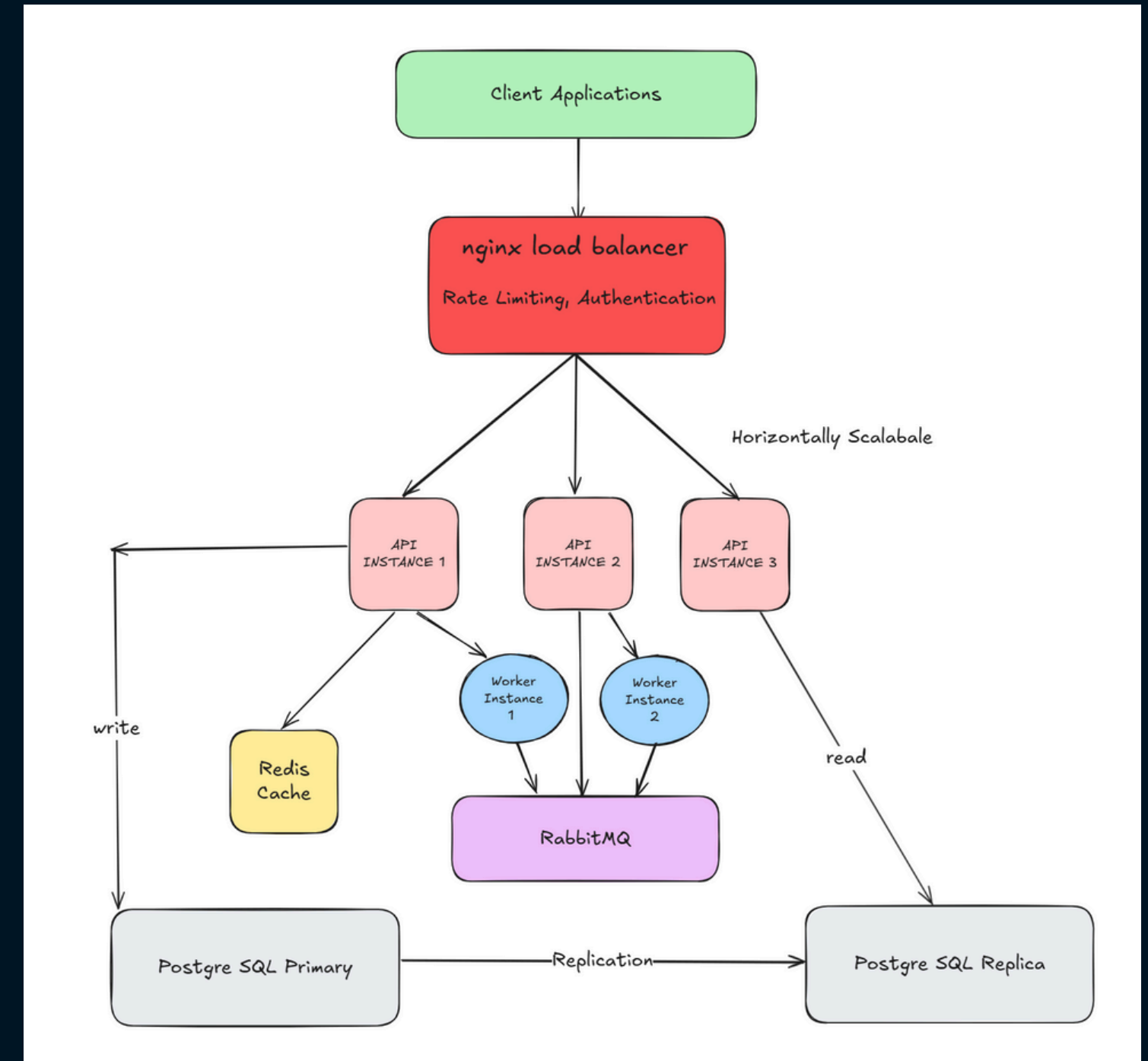
NGINX – The Industry Standard for Load Balancing

🌐 **Used Everywhere:** Powers major platforms like Netflix, Airbnb, Dropbox, and GitHub

Bazaar Inventory System v3 - Final Architecture & Impact

? Key Question: How does this architecture create value?

- ✓ Scales to Thousands of Stores
→ Handles millions of transactions daily
- ✓ Real-Time Inventory Sync
→ Keeps all stores updated instantly
- ✓ Prevents Stockouts
→ Ensures availability → Boosts revenue
- ✓ Reduces Overstocking
→ Less waste & storage cost → Saves capital
- ✓ Data-Driven Decisions
→ Smart purchasing via powerful reports





**This concludes my Case
Study Challenge for
Bazaar! Thankyou for your
time!**

**Explore the code in more
detail and run it on your
device!**